Jetson 엣지 AI 실습 결과 보고서 (1~10단계)

Oct 27, 2026 · @김채림

개요

팀 공용 Jetson에서 GPU 기반 임베딩 모델을 실행하고, 성능 측정, 도메인 미세조정, 물체 검색, 사람 동작·행동 인식까지 1~9단계 실습을 수행했다. 각 단계의 측정 결과와 원인 분석을 정리한다.

항목	내용
장비	Jetson (8GB 메모리, NVMe SSD로 부팅)
소프트웨어	JetPack 7.2.1 (Jetson Linux R39.2.1), Ubuntu 24.04, CUDA 13.2, Python 3.12.3
접속	Mac → Tailscale → `ssh team2@100.80.64.30`, 웹 화면은 `http://100.80.64.30:8000`
계정	메인 계정 team2에 설치 (개인 계정은 t2)
작업 폴더	`~/eg2-lab`, 가상환경 `~/eg2`
카메라	USB 웹캠 1대 (`/dev/video0`)

접속할 때마다 처음에 하는 두 줄:

```
cd ~/eg2-lab && source ~/eg2/bin/activate
```

1~5단계 환경 준비

GPU용 PyTorch가 `+cu132`로 설치되고 실제 GPU 계산까지 성공해서 환경 준비가 끝났다. 자료는 JetPack 6 기준이었지만, 이 Jetson은 JetPack 7.2.1이라 설치 방법을 바꿨다.

단계	한 일	결과
0. SSD	`lsblk`로 확인	이미 SSD로 부팅 중이라 건너뜀 (실수로 만든 `/ssd` 중복 마운트는 되돌림)
1. 버전 확인	`cat /etc/nv_tegra_release`	R39 = JetPack 7. `nvcc`는 없지만 torch 실행에는 필요 없음

단계	한 일	결과
2. 성능 모드	`nvpmodel -m` + `jetson_clocks`	0번은 15W였음 → 목록에서 MAXN_SUPER 번호를 찾아 적용
3. 스왑	`/swapfile` 8GB	`free -h` 에서 스왑 8.0Gi 확인
4. 가상환경	`python3 -m venv ~/eg2`	시스템 파이썬과 분리, team2 계정에만 설치
5. PyTorch	`pip install --pre torch torchvision --extra-index-url https://download.pytorch.org/whl/cu132`	`+cu132`, `(8, 7)`, GPU 행렬 계산 성공

참고 사항:

- 가상환경으로 시스템 파이썬과 분리해, 다른 계정과 상시 실행 중인 프로그램에 영향을 주지 않도록 했다.
- `+cu132`는 CUDA 13.2용 빌드를 뜻한다. `is_available()` 외에 실제 GPU 행렬 연산으로 동작을 검증했다.
- 실행 시 출력되는 CC 8.7 경고는 NVIDIA 안내에 따라 무해한 경고로 판단했다.
- 이후 패키지 설치 시 `-c ~/torch-lock.txt`로 torch 버전을 고정해 덮어쓰기를 막았다.

6단계 성능 측정

텍스트·비전·오디오를 모두 올린 전체 구성도 8GB Jetson에서 메모리 부족 없이 돌아갔다. 가장 무거운 전체 구성의 로드 후 RAM은 약 5,929MB였다.

구성	로드 크기	로드 후 RAM (MB)	입력	지연 (ms)	GPU 사용률 (%)
텍스트 전용	270M	약 4,670	질의 2개	158	약 4
+비전	440M	약 5,463	이미지 5장	2,645	99
+비전	440M	약 5,463	동영상 10초	2,020	99
+오디오	570M	약 5,029	WAV 2개	1,262	약 19~32
전체	740M	약 5,929	혼합	6,000	99

측정 조건: MAXN_SUPER + `jetson_clocks`, 스왑 8GB, 지연은 두 번째 실행 값.

비고:

- **로드 후 RAM**은 `python -i`로 모델을 띄워 둔 `>>>` 상태에서 tegrastats로 읽었다. 아무것도 안 돌릴 때도 약 3,100~3,300MB를 쓰므로(팀원 프로그램, 화면 등) 모델이 실제로 차지한 양은 텍스트 약 1.4GB, 전체 약 2.7GB다.
- **텍스트 GPU 약 4%**는 처리가 0.15초로 너무 짧아 0.1초 간격 측정에 순간만 잡힌 값이다.

- **오디오**는 WAV가 2개라 표 조건(3개)과 다르다.
- **전체 지연 6,000ms**에는 처음 실행되는 텍스트의 준비 시간(642ms, 단독일 때는 158ms)이 섞여 있다.
- 로드 크기는 오디오가 더 큰데 RAM은 비전이 더 높았다. 영상 해독과 이미지 처리에 추가 메모리를 쓰기 때문으로 보인다.

7단계 도메인 미세조정

강의실 데이터로 약 64초 학습하자 accuracy@1이 0.85에서 0.9로 올랐다. 평가 질문 20개 중 1등으로 정답을 찾은 개수가 17개에서 18개로 늘었다는 뜻이다.

모델	accuracy@1	accuracy@3	ndcg@10	학습 시간
기준 (google/embeddinggemma-2)	0.85	1.0	0.938	-
미세 조정 (eg2-classroom)	0.9	1.0	0.95	약 64초 (63.98초)

데이터: 강의실 환경 문서 20개(환기·CO2, 냉난방, 습도, 조명, 소음, 미세먼지, 센서, 안전 등)마다 질문 4개를 만들어 3개는 학습용(`train.jsonl` 60줄), 1개는 평가용(`eval.jsonl` 20줄)으로 나눴다. 평가 질문은 학습 때 본 적 없는 표현이다.

학습 과정: loss가 3.077 → 3.938 → 2.81 → 2.242 → 2.439 → 2.008로 오르내리며 전체적으로 줄었다. 데이터가 60줄로 작아서 흔들림이 있었다.

해석:

- accuracy@3은 학습 전에도 1.0이라 정답이 항상 3위 안에 있었다. 그래서 효과는 accuracy@1과 ndcg@10(정답을 더 위쪽에 놓는 정도)에서 보였다.
- 기준 모델은 Google이 이미 학습한 모델을 받아 쓴 것이라 학습 시간이 없다.
- 데이터가 작아서 향상 폭도 작다. 데이터를 늘리면 차이가 더 분명해질 수 있다.

8단계 물체 검색 (yolo / relabel / yoloe)

YOLO가 모르는 물체 3개(세탁기, 전지 포장, 건전지)를 yolo는 하나도 못 맞혔고, relabel이 가장 많이(3개) 바로잡았다. 세 방식의 속도와 메모리는 큰 차이가 없었다.

세 방식의 차이:

- **yolo**: YOLO가 배운 80종 중 하나를 반드시 고른다. 모르는 물체는 비슷한 클래스로 틀린다.
- **relabel**: YOLO 박스는 그대로 쓰고, 잘라 낸 사진을 임베딩해서 LABELS 목록 중 가장 비슷한 이름을 다시 붙인다.

- **yoloe**: 처음부터 LABELS의 영어 이름을 찾을 대상으로 지정해서 검출한다.

8-4 결과 비교

물체	yolo 라벨	relabel 라벨 (점수)	yoloe 라벨	비고
컵	cup 0.70	컵 (0.652)	cup 0.84	정상 인식
노트북	laptop 0.92	노트북 (0.711)	laptop 0.79	정상 인식. relabel에서 모니터로 바뀌는 경우 있음
휴대폰	cell phone 0.93	휴대폰 (0.73)	cell phone 0.87	정상 인식
드럼 세탁기	microwave 0.79	드럼 세탁기 (0.67)	drum washing machine 0.60	yolo 오인식, relabel·yoloe 정상. relabel은 장면에 따라 전자레인지로도 나옴
원통형 전지 포장	cup 0.67	원통형 전지 포장 (0.731)	battery pack 0.33	yolo 오인식, relabel 정상. yoloe는 장면에 따라 cup으로도 나옴
건전지	book 0.54	건전지 (0.733)	cell phone 0.52	relabel만 정상

모드	검출 fps	임베딩 ms/개	RAM (MB, tegrastats)
yolo	16.7	594	약 5,700
relabel	14.4~17.0	581~614	약 6,026
yoloe	15.0	630	약 5,842

비고:

- 영상 박스 위 숫자는 **YOLO 검출 신뢰도**, relabel 점수는 웹 "최근 추출" 카드의 괄호 속 **임베딩 유사도**다. 둘은 다른 숫자다.
- 검출 fps와 임베딩 ms는 웹 "최근 추출" 줄 값으로 통일했다.
- relabel의 한계: 박스가 안 잡히면 고칠 수 없고, 목록에 없는 물체(충전기 → 물병, 방열판 → 전지 포장)도 억지로 바꾼다. 세탁기처럼 목록 안의 비슷한 짝(전자레인지)과 헷갈리기도 한다.
- yoloe의 한계: 기본 신뢰도 기준(0.25)이 낮아 오검출이 많고(인형 → 마우스 등), 같은 물체도 장면마다 결과가 흔들렸다.

정리: 추가 학습 없이 정확도가 가장 높았던 relabel이 강의실 서비스에 적합하다. 다만 헷갈리는 짝(세탁기/전자레인지 등)은 LABELS 문구를 보강해야 한다.

9단계 사람 동작·행동 인식

규칙에 시간 정보를 더하자(smooth) 깜빡임이 줄었고, 직접 녹화한 데이터로 GRU 모델을 학습하자 규칙으로 불가능했던 박수·팔짱을 인식했다. 학습 모델은 처음 본 사람에게도 15번 중 13번(87%)을 맞혔다.

관절 찾기는 모두 YOLO11n-pose가 하고, 달라지는 건 **판정 방법**뿐이다.

판정 방법	방식	실행
raw (9-1)	한 프레임만 보고 규칙으로 판정	`--raw`
smooth (9-2)	0.6초 다수결 + 1.5초 손목 움직임으로 손흔들기 판정	옵션 없음
행동 규칙 (9-5)	정면 시간·이동 속도로 지나감/힐끗 봄/관심 있음/머무름 판정	`--behavior`
학습 모델 (9-6)	녹화 데이터로 학습한 GRU가 약 2초 움직임을 보고 판정	`--model ...`

9-4 결과 비교 (raw vs smooth)

동작 (실제 횟수)	9-1 raw 카운트	9-2 smooth 카운트	비고 (오판·미검출)
만세 (3회)	만세 3	만세 3	raw: 팔을 올리고 내릴 때 한팔들기 6번 깜빡임. smooth: 깜빡임 없이 정확
한팔들기 (3회)	한팔들기 3	한팔들기 3	raw·smooth 모두 정확 (smooth는 서서 했을 때)
손흔들기 (3회)	0	손흔들기 3	raw: 모두 한팔들기로 인식. smooth: 흔들기 시작 후 1~1.5초 동안 한팔들기로 나오다가 손흔들기로 바뀜. 두 사람이 겹쳐 있을 때 손흔들기로 오판
악수 (3회)	약 5~6	악수 5	공통: 전신이 보여야 인식되고, 맞잡고 가만히 있으면 손목이 가려져 미검출. raw: 크게 흔들 때 켜졌다 꺼졌다 하며 연속 카운트, 두 사람이 겹치기만 해도 악수로 오인식. smooth: 천천히 흔들어도 3회보다 많이 카운트
측면에서 만세 (3회)	만세 6	만세 5	공통: 완전한 옆모습에서는 반대쪽 어깨가 가려져 미검출. raw: 비스듬할 때만 인식되며 2배로 과다 카운트. smooth: 한쪽 방향으로 돌리면 한팔들기·손흔들기로 오판

9-5 행동 패턴 (규칙)

시나리오 (3회씩)	기대 패턴	맞은 횟수	틀린 경우 나온 패턴
그냥 지나감	지나감	3/3	없음 (한 번은 추적이 끊겨 2명으로 기록됐지만 둘 다 지나감)
걸으며 2초 봄	힐끗 봄	1/3	지나감 2회 (2초 봤지만 정면 시간이 1초로만 잡힘)
멈춰서 5초 봄	관심 있음	3/3	없음 (정면 5~6초로 안정적으로 잡힘)
옆을 보며 15초 서 있음	머무름	0/3	힐끗 봄 3회 (옆모습이라 한쪽 어깨가 가려져 멈춘 동안 판정이 갱신되지 않음, 들어올 때 정면 2초가 잡혀 힐끗 봄으로 확정)

규칙은 **시선이 아니라 머리 방향**을 본다. 걸으면서 고개를 돌리면 완전히 정면이 되는 시간이 짧아 힐끗 봄을 놓쳤다.

9-6 학습 결과

모델	라벨	학습 / 검증 조각	최고 검증 정확도	헷갈린 부분
동작 (action.pt, window 30)	arms_crossed, clap, idle, wave	845 / 169	0.976	팔짱 30개 중 3개를 손흔들기, 1개를 기본으로 오판
행동 (behavior.pt, window 30, val 0.3)	glance, interested, passing	648 / 151	1.000	지나감 검증 조각이 2개뿐이라 지나감 정확도는 참고 수준

행동 모델은 다섯 번 고쳐서 만들었다.

1. window 80 → 검증 조각 0으로 학습 불가
2. window 40 → 학습은 됐지만 검증 조각이 힐끗 봄 5개뿐 (100%는 의미 없음)
3. window 30, val 0.3 → 세 라벨 모두 검증 조각 생김, 정확도 0.862
4. 지나감 추가 녹화 → 지나감 학습 조각 21→99, 대신 정확도 0.785 (관심 있음이 지나감으로 많이 헷갈림)
5. 관심 있음을 멈춰서 보는 장면만 다시 녹화 → 관심 있음 학습 124→367, 정확도 1.000

지나감 녹화는 평균 2초짜리 구간 85개로 잘게 끊겨 있었다(50프레임 이상 구간은 4개). 화면이 좁아 빨리 지나가고, 옆모습에서 추적이 자주 끊겼기 때문이다.

9-6 비교표 (규칙 vs 학습 모델)

비교 항목	규칙 (9-2/9-5)	학습 모델 (9-6)
손흔들기 인식 (9-4 시나리오)	3/3 인식. 흔들기 시작 후 1~1.5초 동안 한팔들기로 나오다가 바뀜	양손 각 3회(총 6회) 중 3회 인식. 인식될 때는 한팔들기 없이 바로 wave로 잡힘. 한쪽 손은 인식률이 낮음
측면에서 만세	3회 중 만세 5회 + 한팔들기·손흔들기 오판	만세 3회(과다 카운트 없음). 학습하지 않은 동작이라 wave·clap·arms_crossed로 오판
박수·팔짱 (규칙 없음)	불가	박수: 느리게 3/3, 빠르게 0/3(팔짱으로 오판). 팔짱: 4/4(박수 오판 2회). 비슷한 자세라 서로 헷갈림
힐끗 봄 / 관심 있음 구분 (9-5 표)	힐끗 봄 1/3, 관심 있음 3/3	관심 있음 3/4(수정 전), 2/2(모델 라벨 한글 변환 후). 힐끗 봄은 glance↔passing이 번갈아 나와 불안정. 지나감은 4회 중 1회
처음 본 사람에 대한 정확도	규칙은 사람마다 학습하지 않아 같은 기준	15번 중 13번 (87%): 손흔들기 3/3, 팔짱 3/3, 관심 있음 3/3, 힐끗 봄 3/3, 빠른 박수 1/3

비고:

- 모델은 확신이 70% 미만이면 규칙 결과를 대신 보여 준다. 그래서 모델 라벨과 규칙 라벨이 번갈아 섞여 나온다.
- 코드 수정(모델 라벨을 한글로 변환, idle일 땐 규칙 결과 유지) 전에는 요약 줄에 모델 판정이 반영되지 않아, 상태 변화 기록으로 모델 결과를 확인했다.
- 처음 본 사람에게도 87%를 맞혀, 모델이 녹화한 본인에게만 맞춰지지 않았음을 확인했다. 약점은 본인 테스트와 같이 빠른 박수였다.

10단계 카메라 2대 거리 측정 (접근 감지)

두 카메라 어깨 삼각측량(stereo)이 발목 방식(ankle1)보다 약 6배 정확했고(평균 0.10m vs 0.66m), 다리가 가려져도 위치를 잡았다. 10-3~10-5는 가상 시험(합성 영상)으로 진행했다.

10-1 렌즈 보정

카메라	해상도	재투영 RMS	비고
C920 (교육장, 팀)	640x480	0.169px	자동 초점 끄고 다시 보정
맥 FaceTime HD (집)	1280x720	0.343px	20장 중 오차 큰 4장 제외

카메라	해상도	재투영 RMS	비고
갤럭시탭 IP Webcam (집)	1280x720	0.131px	20장 중 1장 제외

두 대 모두 기준(0.5px 이하)을 만족했고, 중심점이 화면 정중앙 근처다. 처음에는 체커보드가 화면 가운데에만 몰려 RMS 1.28px이 나왔고, 화면을 3x3칸으로 나눠 칸마다 고루 찍게 하자 0.343px로 줄었다.

10-2 카메라 위치 보정 (체커보드 방식)

줄자 대신 두 카메라에 동시에 보이는 체커보드 하나로 위치·방향을 구했다(`calib_board.py`). 재투영 오차는 맥 0.15px, 탭 0.19px, 계산된 두 카메라 간격은 0.468m였고, 삼각측량한 칸 크기가 21.0mm로 입력값과 같아 두 카메라 계산이 서로 맞았다.

10-3 카메라 1대 + 발목 (문제 확인)

발목 키포인트(주황색)는 표시됐지만 거리값이 실제보다 크게 나왔다(약 2.5m 지점에서 6.8~7.2m). 값이 잠깐 나왔다 사라지는 일이 반복됐고, 코드를 고쳐 다시 해도 안정되지 않았다.

- 발목이 화면 아래 끝에 걸려 경계 부근을 발목으로 잡았고, 바닥과 만나는 시선이 거의 수평이 되면서 먼 거리로 계산된 것으로 보인다.
- 10-2에서 체커보드가 조금만 기울어도 바닥 평면이 기울어져 거리가 늘어난다(추정).
- 카메라 가까이 앉아 상체만 보일 때는 사람은 인식되지만 발목이 없어 위치를 못 잡았고, 추적이 끊겨 2초 간격으로 등장·퇴장이 반복됐다.

10-3~10-5 세 방식 비교 — 가상 시험(합성 영상)

시나리오	ankle1 (10-3)	stereo (10-4)	fused (10-5)
트인 곳 걷기 (8m → 1.5m)	38/40 · 0.66m (최대 1.20)	40/40 · 0.10m (최대 0.15)	40/40 · 0.10m (최대 0.15)
책상 뒤 걷기 (다리 가림)	0/30 · —	30/30 · 0.09m (최대 0.12)	30/30 · 0.09m (최대 0.12)
옆으로 비켜 감 (한 카메라에만)	24/32 · 0.55m (최대 0.64)	23/32 · 0.09m (최대 0.11)	23/32 · 0.09m (최대 0.11)

칸 = 위치가 나온 프레임 · 평균 오차(바닥 위치). 가상 시험(합성 영상) — 가상 카메라 2대(1280x720, 초점거리 960px, 높이 1.2m, 간격 1.0m, 10° 아래)가 본 합성 화면에 yolo11n-pose를 실제로 돌렸다. 정답 위치를 알아 오차를 바로 채점했다. 키포인트 신뢰도 0.5, fused 한쪽 추정 높이 1.2m. 2026-11-03.

- 발목 방식은 다리가 가려지면 위치를 전혀 내지 못했고(0/30), 보일 때도 발목 점이 조금만 어긋나면 바닥과 만나는 점이 크게 밀렸다.
- 어깨 삼각측량은 발과 상관없이 두 카메라에서 보이면 위치를 잡고 오차가 0.1m 안팎이다.

- 한 카메라에만 보이는 경우 fused는 stereo보다 더 잡지 못했다(23/32로 같음). 사람이 화면 끝에 걸려 어깨 신뢰도가 기준을 넘지 못한 것으로 보여, fused의 보완 효과는 이 시험으로는 확인하지 못했다.

10-6 접근 알람, 10-8 이동 타겟 — 가상 시험(합성 영상)

고정 타겟 알람은 정답과 0.1초 안에서 울렸고(접근 2.6초 vs 정답 2.6초, 이탈 7.7초 vs 7.6초, 헛알람 없음), 이동 타겟은 빠를수록 충돌 위험 알람이 더 일찍 울렸다.

타겟 속도 (m/s)	추정 속도 (m/s)	충돌 위험 알람 때 거리 (m)	근접 알람보다 먼저 울린 시간 (s)
천천히 0.5	0.49	0.82	−1.2 (늦음)
보통 1.0	1.00	1.94	0.5
빠르게 1.5	1.49	2.89	1.0

가상 시험(합성 영상) — 근접 1.5m(해제 1.7m), 충돌 위험 = 접근 속도 > 0.5m/s 이고 TTC < 2초, 타겟은 다른 사람, 속도는 최근 1초 위치의 직선 기울기. 2026-11-03.

- 0.5m/s는 접근 속도 기준(0.5m/s)과 거의 같아 충돌 위험이 늦게 울렸다. 실습 안내문의 '천천히는 알람이 안 울릴 수 있음'과 같은 결과다.
- 가상 시험에서는 타겟을 YOLOE로 찾는 단계 대신 사람 두 명을 두 카메라에서 짝지어 위치를 구했다. 두 카메라 시간 차가 0이라, 실제 웹캠(약 80ms)에서는 1m/s에 약 8cm 오차가 더 생긴다.
- 10-7 비교표는 위 10-3~10-5 가상 시험 표로 대신한다.

문제 해결 기록

대부분의 오류는 코드 문제가 아니라 환경(버전, 다른 프로그램, 입력 방법) 문제였다.

증상	원인	해결
`torchaudio`가 CUDA 13.0, torch가 13.2라 import 에러	다른 패키지를 깔면서 짝이 안 맞는 torchaudio가 같이 설치됨	`pip uninstall -y torchaudio` (LLM에는 불필요)
영상 임베딩에서 `torchcodec` 없음, 이후 `libswscale.so.9` 없음	Jetson 기본 FFmpeg가 기능을 줄인 버전	`torchcodec --no-deps` 설치 + 우분투 FFmpeg 6 부품 추가 설치
`ffmpeg`로 RTSP 녹화·libx264 안 됨	Jetson FFmpeg에 네트워크와 libx264가 빠져 있음	`imageio-ffmpeg`의 완전한 ffmpeg(`$FF`) 사용, USB 카메라는 `-f v4l2 -i /dev/video0`
파이썬 코드를 터미널에 붙여 넣어 문법 에러	bash는 파이썬 문법을 모름	`python << 'EOF' ... EOF`로 감싸거나 파일로 저장

증상	원인	해결
`nano: 명령어를 찾을 수 없음`	nano 미설치	`sudo apt install -y nano` 또는 `cat > 파일 << 'EOF'`
`EOF` 를 쳐도 안 끝남 (`⊏EOF`)	키보드가 한글 상태	영어로 바꾸고 다시 `EOF` 입력
`ModuleNotFoundError: torch`	가상환경 밖에서 실행	`source ~/eg2/bin/activate` 후 실행
웹 페이지는 열리는데 0.0 fps, 영상 없음	`웹캠을 열 수 없음` (다른 앱이 카메라 사용 중)	`sudo fuser -v /dev/video0` 로 확인 후 `pkill -f pose_app.py`
`Ctrl + C` 로 앱이 안 꺼짐	브라우저 영상 연결이 열려 있음	탭을 먼저 닫거나 `Ctrl + \` , 안 되면 새 창에서 `pkill`
앱이 켜진 창에 명령을 쳐도 실행 안 됨	이전 앱이 아직 실행 중	이전 앱을 끈 뒤 프롬프트가 보일 때 입력
SSH `Operation timed out`	Jetson이 Tailscale에서 오프라인(회색)	Jetson 전원·네트워크 확인, USB-C 직결(`192.168.55.1`) 대안
행동 학습 `윈도가 부족함`	지나감·힐끗 봄 구간이 짧아 검증 조각 0	`--window 30 --val 0.3` 으로 줄임
YOLOE 실행 시 CLIP 자동 설치 후 경고	설치 직후 반영 안 됨	앱을 다시 실행

배운 점

가장 크게 배운 점은 **데이터의 양만큼 깨끗함이 중요하다**는 것이다. 관심 있음 녹화에서 걸어 나가는 장면만 빼자 정확도가 22%에서 100%로 올랐다.

- **환경 버전이 먼저다.** JetPack 7에서는 자료(JetPack 6 기준)와 설치 방법이 달랐고, torch 버전을 고정해야 다른 패키지가 덮어쓰지 않았다.
- **점수는 시험 문제 구성을 봐야 한다.** 검증 조각이 한 라벨에 몰리면 100%나 78.5% 같은 숫자는 의미가 없다. 라벨별 혼동 행렬이 더 정확하다.
- **규칙은 정한 기준 밖에서 틀린다.** 측면, 짧은 고개 돌림, 가려진 관절처럼 기준값이 예상하지 못한 상황에서 오판이 났다.
- **학습 모델은 배운 것만 안다.** 학습하지 않은 만세는 아는 동작 중 하나로 억지로 분류했고, 비슷한 자세(박수↔팔짱, 지나감↔힐끗 봄)는 서로 헷갈렸다.
- **녹화와 테스트 조건을 맞춰야 한다.** 앉은 자세, 거리, 손 방향이 녹화 때와 다르면 인식률이 떨어졌다.
- **진짜 성능은 실시간·낯선 사람 테스트로 본다.** 스크립트의 검증은 같은 녹화의 뒷부분이라 높게 나온다.